FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering
Academic Year 2025-26

Rubrics for Lab Experiments

Class : B.*E. Computer Engineering* **Subject Name :*BDA Lab***

Semester : VII **Subject Code :CSL702**

Practical No:	7
Title:	Data stream Algorithm: Implement DGIM Algorithm using Map reduce.
Date of Performance:	18/9/2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation:

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorr ect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

Experiment No 7

Aim: Data stream Algorithm: Implement DGIM Algorithm using Map reduce.

Theory: each bit of the stream has a timestamp, the position in which it arrives. The first bit has timestamp 1, the second has timestamp 2, and so on.

Since we only need to distinguish positions within the window of length N , we shall represent timestamps modulo N, so they can be represented by log N bits. If we also store the total number of bits ever seen in the stream (i.e., the most recent timestamp) modulo N , then we can determine from a timestamp modulo N where in the current window the bit with that timestamp is.

We divide the window into buckets, 5 consisting of:

1. The timestamp of its right (most recent) end.
2. The number of 1's in the bucket. This number must be a power of 2, and we refer to the number of 1's as the size of the bucket.

here are six rules that must be followed when representing a stream by buckets.

- The right end of a bucket is always a position with a 1.
- Every position with a 1 is in some bucket.
- No position is in more than one bucket.
- There are one or two buckets of any given size, up to some maximum size (r).
- All sizes must be a power of 2.
- Buckets cannot decrease in size as we move to the left (back in time).

Objective:

- Understand Stream Mining DGIM algorithm(to count distinct no of elements in the stream)
- Write python program to implement DGIM algorithm using map reduce.

Input:

Stream.txt

111101101010110

Output:
Size of bucket: 4

Size of bucket: 4

Size of bucket: 2

Result: Implementation of the solution for above problem and attach input, program, intermediate and output files.

lab-6 > mapper.py > ...

```python
# mapper.py
import sys

def mapper(stream):
    timestamp = len(stream)
    for i in range(timestamp - 1, -1, -1):
        if stream[i] == '1':
            print(f"{i+1}\t1")  # Emit timestamp and 1

if __name__ == "__main__":
    with open("stream.txt", "r") as f:
        stream = f.read().strip()
    mapper(stream)
```

```python
# reducer.py
import sys
from collections import deque

def reducer():
    buckets = deque()
    max_bucket_size = 0

    for line in sys.stdin:
        timestamp, val = line.strip().split("\t")
        timestamp = int(timestamp)
        val = int(val)

        if val == 1:
            buckets.appendleft((1, timestamp))  # (size, timestamp)
            compress_buckets(buckets)

    # Output all buckets
    for size, ts in buckets:
```

```
        print(f"Size of bucket: {size}")

def compress_buckets(buckets):
    i = 0
    while i < len(buckets) - 2:
        if buckets[i][0] == buckets[i+1][0] == buckets[i+2][0]:
            # Merge the last two (older ones)
            size = buckets[i+1][0] * 2
            ts = buckets[i+1][1]  # use more recent timestamp
            del buckets[i+2]
            del buckets[i+1]
            buckets.insert(i+1, (size, ts))
            i = 0  # restart
        else:
            i += 1

if __name__ == "__main__":
    reducer()
```

lab-6 > stream.txt

```
1  ///stream.txt
2  111101101010110
3
```

Conclusion:

- DGIM provides an efficient way to **estimate recent activity** in large data streams with **logarithmic space**.
- MapReduce allows scalable processing of streaming data.
- This experiment reinforces understanding of **stream processing**, **approximate algorithms**, and **distributed computing** principles.